All pairs shortest paths

Definition:

- For the given weighted connected graph (undirected or directed) the all-pairs shortest paths problem find the distance from each vertex do all over other vertices...
- i.e, the algorithm finds lengths of the shortest paths from each vertex to all other vertices.

Floyd's Algorithm

ALGORITHM Floyd (W[1..n, 1..n])

// Implement Floyd's algorithm for the all-pairs shortest-paths problem.

// Input: The weight matrix W of a graph with no negative-length cycle.

// Output: The distance matrix of the shortest paths' lengths.

```
D ← W    // is not necessary if W can be overwritten
for k ← 1 to n do
    for i ← 1 to n do
        for j ← 1 to n do
            D[i,j] ← min{D[i,j], D[i,k] + D[k,j]}
return D.
```

3 a

Warshall's Algorithm

- The Warshall algorithm is used to find the existance of the path between all the pair of the vertices in a given weighted connected graph.
- It is applicable to both directed & undirected graph. to find transitive closure we can use DFS or BFS.
- The transitive closure of a directed graph with n vertices can be defined as nxn boolean matrix.
  $T = \{t_{ij}\}$, In which the element in the i^{th} row & j^{th} column is 1, if there exist a non-pridial path from the vertex to the j^{th} vertex otherwise, $t_{ij} = 0$.
- Steps
  1. Generate the adjacency matrix.
  2. Consider the path through vertex A.
  3. Consider the path through vertex B.
  4. Consider the path through vertex C.
  5. Consider the path through vertex D.

3 b

Warshall Algorithm

ALGORITHM Warshall (A[1..n, 1..n])

// Implements warshall's algorithm for computing the transitive closure.
// Input : The adjacency matrix A of a diagraphic n vertices.
// Output : The transitive closure of the diagraph.

Dijkstra's Algorithm

- It is a single source shortest path
- It finds the shortest path from source to all other vertices in the given graph & it is similar to the prim's algorithm.
- We can apply this algorithm both directed & undirected graph with non-negative weights only.
- It finds the shortest path to the vertices of the graph in the order of their distance from a given source vertex.
- First it finds the shortest path from the source to a vertex nearest to it. then to a second nearest and so on.

ALGORITHM Dijkstra (G, s)

// Dijkstra's algorithm for single-source shortest paths.
// Input : A weighted connected graph $G = \langle V, E \rangle$ with non-negative weights and its vertex s.
// Output : The length d_v of shortest path from s to v and its penultimate vertex p_v for every vertex v in V

Initialize (Q) // initialize priority queue to empty for every vertex v in V
    $d_u \leftarrow \infty$; $p_u \leftarrow$ null
    Insert (Q, v, d_v) // Initialize vertex priority in the priority queue.

$d_s \leftarrow 0$; Decrease (Q, s, d_s) // update priority of s with d_s
$V_T \leftarrow \emptyset$
for i ← 0 to |V| - 1 do
    $u^* \leftarrow$ DeleteMin (Q) // delete the minimum priority element
    $V_T \leftarrow V_T \cup \{u^*\}$

4th continued part

for every vertex u in $V - V_T$ that is adjacent to u^* do

if $du^* + \omega(u^*, u) < du$

$du \leftarrow du^* + \omega(u^*, u)$; $P_u \leftarrow u^*$

Decrease (Q, u, du)

Topological Sorting.

- A topological sort of a DAG (Directed Acyclic Graph) $G = V, e$ is a linear ordering of all its vertices such that if G contains an edge u, v then u appears before v in the ordering.
- If the graph forms a cycle, then no linear ordering is possible.
- Topological sort of a graph can be viewed as an ordering of its vertices along a horizontal line, so that all the directed edges go from left to right.
- This topological ordering has 2 methods mainly DFS method and source removal method.
- In our program we have used source removal method which belongs to the decrease and conquer method.

Design steps to the source removal method:

1. In a given graph identify the vertex with no incoming edges (indegree = 0) and delete along with the outgoing edges.
2. If there are several vertices with no incoming edges breaks the tie Arbitrarily.
3. The order in which the vertices are visited and deleted one by one results in the topological ordering.

- Equation

$$v[i,j] = \begin{cases} 0, & \text{if } i=j=0 \\ v[i-1, j], & \text{if } \omega i > j \\ \max(v[i-1, j], v[i-1, j-\omega i] + vi), & \text{if } \omega i \leq j \end{cases}$$